Doc-Genie: Professional Python Function Docstring Generator with AST Analysis

Abstract

Doc-Genie is an intelligent Python documentation generation platform designed to automate the creation of professional docstrings for Python functions. The system leverages Python's Abstract Syntax Tree (AST) analysis to understand function structure, parameters, return types, and logical operations. By combining AST parsing with an interactive Gradio-based user interface, the platform simplifies the process of maintaining consistent and accurate documentation in Python projects. The tool supports multiple documentation formats such as Google and NumPy style docstrings and offers export features including PDF and TXT reports. Doc-Genie significantly improves software maintainability, readability, and developer productivity by transforming manual documentation tasks into an automated workflow.

Introduction

Software documentation is an essential component of software development. It helps developers understand the purpose and functionality of code and ensures that systems remain maintainable and scalable. However, writing documentation manually can be time-consuming and is often overlooked by developers.

Doc-Genie addresses this challenge by providing an automated documentation generator that analyzes Python functions and produces structured docstrings. The system integrates code analysis, intelligent pattern detection, and documentation generation in a single platform.

The project aims to improve code quality and encourage developers to follow industry-standard documentation practices by making the process fast, simple, and efficient

Objectives

1. Automate the generation of Python function documentation.
2. Use AST analysis to understand code structure and behavior.
3. Provide professional docstring formats such as Google and NumPy styles.
4. Build a user-friendly interface for developers using Gradio.
5. Enable export of documentation in PDF and TXT formats.
6. Improve code readability, maintainability, and collaboration.
7. Demonstrate the use of static code analysis for documentation automation.

Literature Review

Modern software systems often rely on documentation standards such as Google Docstrings, NumPy Docstrings, and reStructuredText formats. Several documentation tools exist, including Sphinx, Doxygen, and MkDocs. However, these tools require developers to manually write documentation.

Recent research in software engineering focuses on automating documentation through static analysis and machine learning. Abstract Syntax Tree (AST) parsing is widely used for code analysis

because it represents the structure of source code in a hierarchical tree format.

Doc-Genie combines AST-based analysis with automated docstring generation, providing a lightweight and interactive solution for developers.

Problem Statement

Many software projects suffer from poor documentation practices. Functions may lack descriptions, parameter explanations, and return value information. This leads to difficulties in code understanding and maintenance.

Manual documentation is often neglected due to time constraints. Therefore, an automated system that can analyze source code and generate documentation is highly valuable.

Doc-Genie aims to solve this problem by automatically generating structured documentation based on the actual code implementation.

System Overview

Doc-Genie is designed as a modular application consisting of several functional components. The system receives Python source code as input and processes it using AST parsing techniques.

The extracted information is then analyzed to understand function behavior and generate context-aware documentation. The generated docstring is displayed through an interactive web interface where users can copy, download, or share the output.

The system focuses on accuracy, usability, and extensibility, allowing future integration with development tools and IDEs.

System Architecture

The architecture of Doc-Genie consists of the following major components:

1. Code Input Module – accepts Python code or uploaded files.
2. AST Analysis Engine – parses the code and extracts structural information.
3. Logic Analysis Module – identifies loops, conditions, and operations.
4. Docstring Generator – produces formatted documentation.
5. User Interface Module – provides interactive controls using Gradio.
6. Export Module – allows users to download reports in PDF or TXT format.

These modules work together to transform raw Python code into professional documentation.

Technologies Used

Programming Language: Python 3.8+

Libraries:
- AST (Abstract Syntax Tree)

- Gradio
- ReportLab
- Transformers
- Torch
- Requests
- Black
- Autopep8

Development Platform:
- Google Colab

Web Technologies:
- HTML5
- CSS3

These technologies provide a powerful combination of code analysis, machine learning capabilities, and user-friendly interface development.

Implementation

The implementation of Doc-Genie involves multiple stages.

First, the system parses the Python code using the AST module. The parser identifies function definitions and extracts metadata such as parameter names, types, default values, and return types.

Next, the logic analysis module examines the function body to detect loops, conditional statements, mathematical operations, and function calls.

Based on this analysis, the docstring generator creates structured documentation following the selected style format.

User Interface

The system uses Gradio to create an interactive web interface. The interface includes:

- Code input text area
- File upload option
- Style selector (Google / NumPy)
- Generate documentation button
- Output display area
- Export buttons for PDF and TXT

This interface allows developers to quickly analyze code and generate documentation without installing complex software tools.

Testing

The system was tested using multiple Python functions with varying levels of complexity. Test cases included simple mathematical functions, recursive functions, and functions with multiple parameters.

The generated documentation was evaluated for accuracy, completeness, and formatting consistency.

Results indicated that the system successfully generated appropriate docstrings for most functions, demonstrating the reliability of AST-based analysis.

Advantages

1. Reduces manual documentation effort.
2. Ensures consistent documentation style.
3. Improves code readability.
4. Enhances collaboration among developers.
5. Provides export and sharing features.
6. Supports multiple documentation standards.

Limitations

1. Currently supports only Python functions.
2. Complex logic may require manual refinement of generated descriptions.
3. Limited integration with development environments.

Future Enhancements

Future versions of Doc-Genie may include:

1. Support for additional programming languages.
2. Integration with IDEs such as VS Code and PyCharm.
3. Machine learning models for more advanced code understanding.
4. Automatic documentation publishing to GitHub.
5. Support for class-level and module-level documentation.

Conclusion

Doc-Genie demonstrates the effectiveness of combining static code analysis with automated documentation generation. By using AST parsing and intelligent logic detection, the system provides accurate and professional docstrings with minimal developer effort.

The project highlights the importance of automation in software engineering and shows how developer productivity can be improved through intelligent tools. With further enhancements, Doc-Genie can evolve into a comprehensive documentation platform for modern software development.